

Nutanix Prep

Virtualization & Cloud Essentials

→ **Virtualization** is a fundamental technique that creates virtual (rather than physical) versions of computing resources such as servers, storage devices, network, applications and operating systems. This is achieved through a software known as hypervisor which abstracts hardware resources and allow multiple virtual environments or machines to run independently on a single piece of physical hardware.

→ Virtualization allows **multiple operating systems** and **applications** to run **concurrently** on the same physical hardware, thereby maximizing **resource utilization and flexibility**.

→ Types of Virtualization : Hardware Virtualization, OS virtualization, Server Virtualization, Storage Virtualization, Storage Virtualization, Network Virtualization.

Type	Description	Key Benefits	Example Use Case
Server Virtualization	Multiple virtual servers on one physical server	Resource efficiency, cost savings	Web/app hosting in data centers
Storage Virtualization	Pooling multiple storage devices into one logical storage unit	Simplifies management, better utilization	Enterprise backup & recovery
Network Virtualization	Virtual networks & devices over a single physical infrastructure	Flexible management, enhanced security	Cloud networking, segmentation
Desktop Virtualization	Remote access to centralized desktop environments	Supports remote work, streamlined updates	VDI for remote workers
Application Virtualization	Running apps separate from OS/hardware	Portability, easier updates, isolation	SaaS, legacy app support
Data Virtualization	Unified data access across sources	Real-time integration, analytics	Business intelligence platforms

Type	Description	Key Benefits	Example Use Case
GPU Virtualization	Shared GPU resources among virtual machines	Accelerates intensive computing tasks	AI model training, graphics

→ A **Hypervisor** also known as a **virtual machine monitor (VMM)**, allocates computing resources such as CPU, memory and storage from the host to multiple VMS. Each VMs operates independently with its own operating systems and applications, effectively simulating a complete computer environment. The hypervisor manages these **resources dynamically**, ensuring that each VM receives the necessary allocation while maintaining isolation from other VMs.

→ There are two types of hypervisors :

1. Type-1 (Bare - metal)

- **Directly runs on the host's hardware** : Type1 hypervisors operate directly on the physical hardware without needing a host operating system. This allows for better performance and resource management.
- Examples : VMware EsXi, Microsoft Hyper-V, Citrix XenServer

2. Type-2 (Hosted)

- **Runs on top of a host operating system** : Type2 hypervisors are installed as software applications on an existing operating system, they rely on the host OS for resource management.
- Examples : Oracle virtualbox, VMware workstation

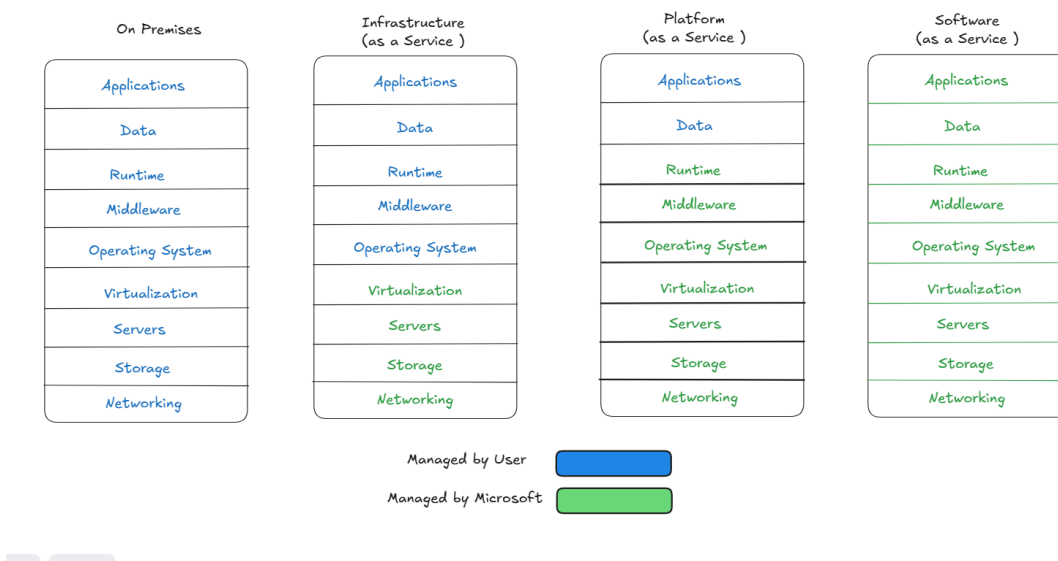
→ **Advantages** of cloud computing : Reduced Capital Expenditure (Capex), Lower maintenance costs, Scalability, Speed & agility, Flexibility, Mobility, Disaster recovery, Enhanced security

→ **Disadvantages** of Cloud computing : Downtime, security, vendor lock-in, vulnerable to attacks, limited control & flexibility, latency issues, technical issues, varied performance, bandwidth constraints.

→ IaaS is like **renting a house**, provides on-demand, scalable virtualized computing resources over the internet such as server, storage, networking and virtualization, users manage os, applications & data while the provider manages the underlying physical infrastructure. ex : aws, gcp

→ PasS is like **renting a house and adding infrastructure**, it offers a cloud platform with hardware & software tools delivered over the internet for application development and deployment, limited control over the environment and applications. ex : google app engine, azure

→ SasS is like **renting a fully furnished flat**, access to software applications via the internet, minimal control over user settings, provider manages everything including applications, very limited customization. ex : salesforce, google workspace, microsoft 365.



Public Cloud : Cloud resources (servers, storage, applications) are owned and operated by third-party providers and delivered over the internet, resources are shared among multiple organizations (multi-tenant), highly scalable, cost-effective with pay-as-you-go pricing. ex : aws, azure, gcp.

Private Cloud : Cloud infrastructure is dedicated to a single organization, can be hosted on-premises or by a service provider, provides greater control, customization and security, typically more expensive due to dedicated resources.

Hybrid Cloud : Combines both public and private clouds, enabling data and applications to move between them, balances flexibility, cost efficiency, scalability and security.

Community Cloud : Shared cloud infrastructure among several organizations with common concerns (ex : security, compliance), can be managed internally or by a third party, ideal for collaborative projects or industries with shared regulatory requirements.

Multi Cloud : The use of multiple cloud services from different providers simultaneously, avoids vendor lock-in, improves redundancy and optimizes services for different workload needs, requires managing complexity across diverse platforms.

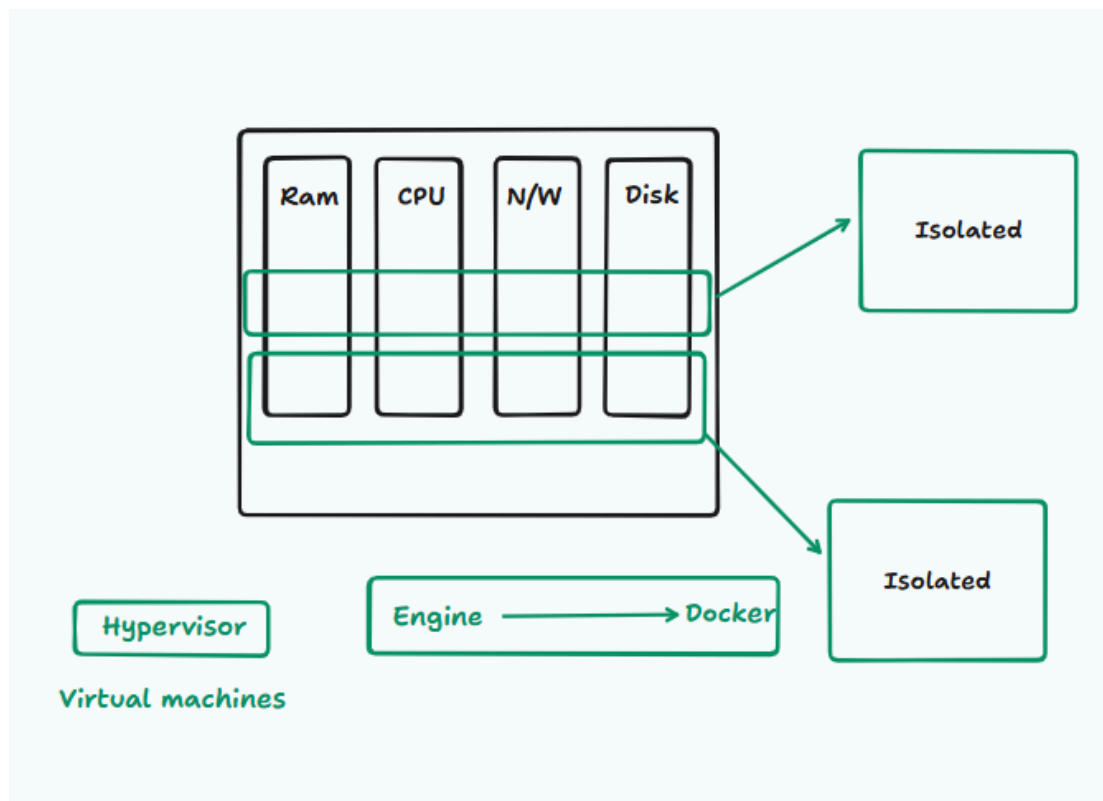
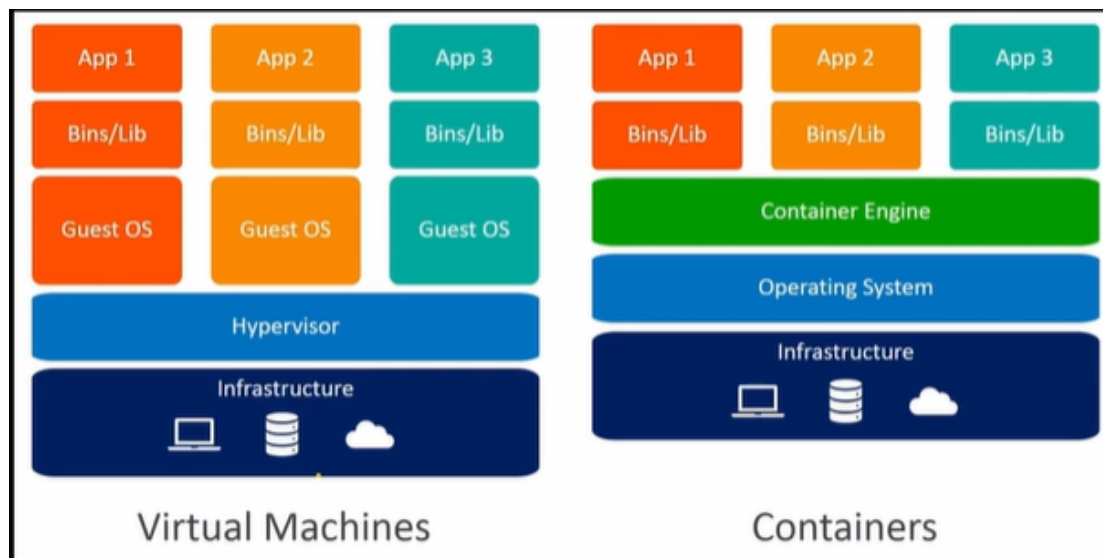
→ These models offer different trade-offs in terms of control security, cost and scalability depending on organizational needs. Hybrid and multi-cloud approaches are increasingly popular for their flexibility and risk mitigation.

→ A **virtual machine (VM)** is a software based emulation of a physical computer. It includes its own virtualized CPU, memory, storage, and network interface and runs its own operating system and applications. These are best for entire OS level separation or support for multiple operating systems.

→ **Thin provisioning** allocates storage space dynamically to a virtual machine as data is written, the virtual disk size appears large to the OS, but actual physical storage is allocated only when needed.

→ **Thick provisioning** allocates all the required physical storage upfront when the virtual disk is created, the entire disk size is reserved regardless of actual usage.

→ A **container** is a lightweight, stand alone, executable software package that wraps an application and all its dependencies (libraries, binaries and config files). All containers share the host system's operating system kernel but are isolated from each other at the process level. Containers are extremely portable, have minimal overhead, and can be spun up or down much faster than VMs. They're ideal for rapid scaling, microservices and cloud native application development. These are ideal for deploying software in lightweight, portable, rapidly scalable environments, especially for microservices architectures.



Resource allocation and scheduling in virtualization are crucial for efficiently sharing the underlying physical hardware among multiple virtual machines (VMs) or containers.

→ The hypervisor dynamically allocates physical resources to each VM based on workload demands.

→ The hypervisor uses algorithms (ex: round robin, priority based, fair share) to decide which vCPU gets access to the physical CPU at any moment. This ensures that VMs get their allotted share and prevents any one VM from monopolizing the CPU.

Virtual Machine (VM) migration refers to moving a VM from one physical host to another. There are two main types

→ **Live Migration (Hot Migration)** : It is moving a running VM from one host to another with little or not downtime. The VM continues to operate and users do not notice any interruption.

- VMs memory pages copied → paused → last pages moved → connections and storage are maintained

→ **Cold Migration (Offline Migration)** : It means powering off (shutting down) the VM before moving it to another host.

- VM is stopped → config files, logs, disks image moved → started on new host

Feature	Live Migration	Cold Migration
VM State	Running (active)	Shut down (inactive)
Downtime	Minimal/nearly zero	Significant (VM unavailable)
Use case	High-availability, critical apps	Planned maintenance, hardware upgrades
Hardware similarity	Usually required	Can migrate across different architectures
Complexity	High (needs shared storage, network sync)	Low (file copy)

→ In summary, live migration is ideal for scenarios where uptime is crucial, while cold migration is appropriate for non-critical VMs or when hardware changes make live migration impossible.

High Availability techniques : **Clustering, automated monitoring & restart, dynamic workload balancing**

Fault Tolerance techniques : **active-passive replication, no service interruption**

→ A **snapshot** is a picture of a VMs state (disk, memory and settings) at a specific point in time. Snapshots let you revert a VM to a previous state if something goes wrong (ex : after updates or tests) . when a snapshot is taken, the current disk state is preserved and all new writes are redirected to a separate file.

→ Cloning creates an exact, independent copy of a VM in its current state. The new VM can run or be modified without affecting its original. Types of cloning : full cloning, linked cloning.

Feature	Snapshot	Clone
Independence	No; always relies on original VM and disk structure	Yes (full clone) or partial (linked clone)
Purpose	Rollback point, temporary safety net	New, reusable, independent VM
Usage	Testing, patching, backups before changes	Mass deployment, distribution, migration
Storage	Uses less space (until many accumulate)	Full clone uses as much as original; linked less
Longevity	Short-term rollback	Long-term use

Storage Efficiency :

→ **Compression** : This technique reduces the size of data by using algorithms to encode it more efficiently, eliminating statistical redundancy. It can be lossless (the original data can be perfectly reconstructed) or lossy (some data is discarded, used for media like images and video)

→ **Deduplication** : This technique eliminates redundant copies of data. It works by analyzing data at the file or block level and storing only one unique instance of each piece of data. Subsequent identical copies are replaced with a small pointer to the original stored instance. Deduplication is extremely effective in environments with high data redundancy, such as virtual infrastructure (VDI), where many users have identical operating system files and in backup systems, where daily backups contain large amounts of unchanged data.

→ **Elasticity** in cloud computing refers to the ability of a system to dynamically provision and de-provision computing resources automatically in response to changing workload demands

→ **Cloud Orchestration** involves coordinating and managing multiple automated tasks into a complete workflow or process, goes beyond automation by linking interdependent tasks across different systems or cloud services,

Serverless Computing : A cloud computing execution model where the cloud provider manages the infrastructure, including server provisioning, scaling and maintenance, developers write and deploy code without worrying about the underlying server management.

Function as a Service (FaaS) : A subset of serverless computing focused on running individual functions or small pieces of code, functions are executed in response to events (HTTP requests, file uploads, scheduled tasks)

→ Serverless computing abstracts infrastructure management, while FaaS enables running discrete, event-triggered functions on-demand without server administration. Both promote agility, scalability, and cost-efficiency in modern cloud application development.

Edge Computing : Distributed computing architecture that brings data processing, storage and analysis closer to the data source or end-user device. Happens at or near the "edge" of the network, such as IoT devices, local edge servers or gateways.

Cloud computing : Centralized computing resources and services hosted in large remote data centers accessible over the internet. Computations, storage, and services are provided by large-scale data centers managed by cloud providers like AWS, Google Cloud, or Azure

Cluster Management : It refers to tools or frameworks that coordinate and administer a group of connected servers (nodes) that collectively provide computing or storage resources.

→ Distributed storage systems ensure reliable and scalable data storage across multiple nodes by using replication and sharding. Cluster management systems provide the necessary tools to monitor, allocate, and automate the collective operation of these distributed storage resources, making large-scale, resilient infrastructure achievable and efficient.

→ Nutanix USP is the core hyperconverged infrastructure platform that integrates compute, storage and networking resources into a single software-defined solution. It simplifies IT infrastructure by eliminating hardware silos and improves

operational efficiency with features like deduplication, compression and erasure coding, supporting a wide range of workloads.

→ Nutanix's HCI (Hyperconverged Infrastructure) is a software defined infrastructure that combines compute, storage and networking into a single system managed through software like Prism. It allows organizations to scale seamlessly by adding nodes while simplifying operations and reducing complexity. The Nutanix Cloud platform is built on HCI, supporting diverse workloads from virtualized apps to cloud native and big data analytics.

→ AHV (Acropolis Hypervisor) is Nutanix's native virtualization hypervisor. It is modern, secure and designed for hybrid cloud environments. AHV powers virtual machines (VMs) and containers, offering advanced virtualization features such as live migration, high availability and dynamic scheduling. It integrates tightly with the Nutanix HCI stack for streamlined management.

→ Nutanix Prism is the management and control interface for Nutanix environments. It provides a unified, easy-to-use control plane for managing compute, storage and networking, VMs and containers across multiple clusters and clouds. Prism leverages machine learning to automate routine workflows, provide predictive analytics, alert on anomalies and optimize infrastructure performance. It has two components : Prism Element (for local cluster management) and Prism Central (for multi-cluster centralized management).

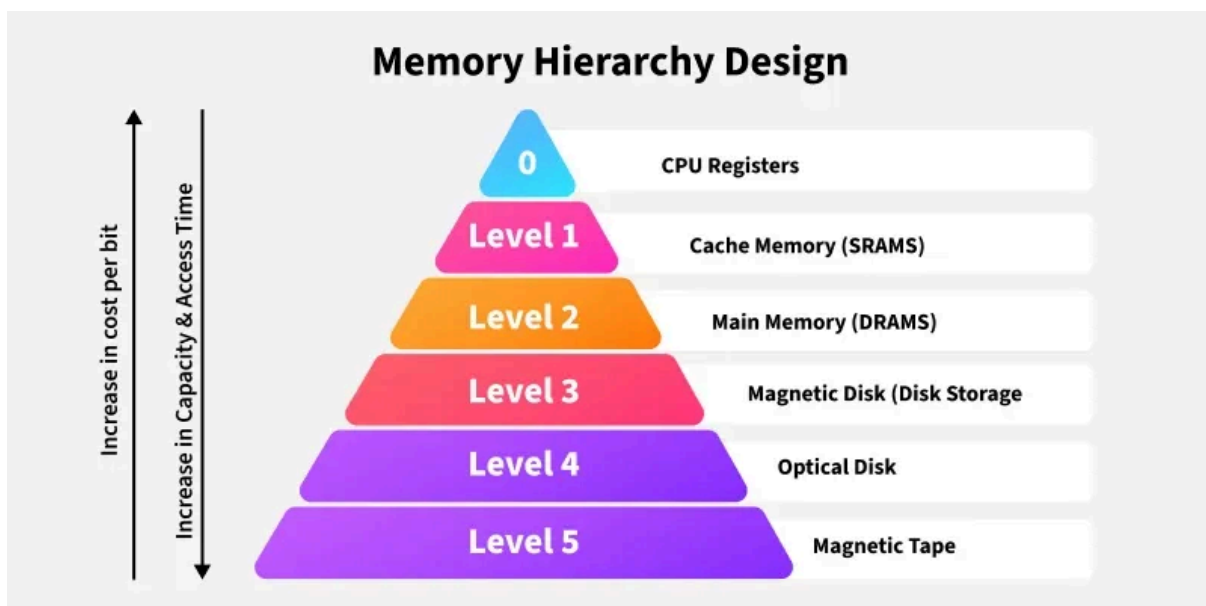
Hardware

→ CPU architecture refers to the design and structure of a Central Processing Unit (CPU) defining how it processes instructions, manages memory and interacts with other hardware. Major aspects include :

- **Instruction Set Architecture (ISA)** : The set of instructions the CPU can execute
- **Microarchitecture** : The implementation details (pipelines, caches, cores) that realize the ISA.
- **Cores / Threads** : Modern CPUs have multiple cores, which can each handle their own computing tasks. Some CPUs feature simultaneous multithreading, allowing each core to handle multiple threads.

- **Cache Hierarchy** : CPUs are fast, small-feeeling memory caches (L1, L2, L3) to reduce wait times for frequently accessed data.
- **Memory Management** : Hardware support for translating virtual memoyr addresses to physical addresses (usually via an MMU - Memory Management Unit).

Feature	Description
ISA	Defines CPU instruction format (e.g., x86, ARM)
Multi-core/Threading	Parallel data processing for improved performance
Caches	Fast, small memories to reduce latency
MMU	Hardware translating virtual to physical memory addresses
VT-x/AMD-V	Hardware extensions for efficient virtualization
SLAT (EPT/RVI/NPT)	Hardware-assisted memory virtualization for faster performance in VMs



Ram : Primary main memory directly accessible by the CPU, sotres data and instructions that are actively used by the CPU during execution, volatile memory, larger capacity than caches but slower access speed.

Cache Memory : small, very fast memory located close to the CPU cores, temporarily holds frequently accessed data and instructions to reduce the time CPU spends fetching from slower main memory.

Virtual Memory : A memory management technique that uses disk storage to extend apparent RAM capacity, allows systems to run programs larger than physical RAM and provides memory isolation between processes. The OS manages virtual memory by mapping virtual addresses used by programs to physical memory addresses, when data is not in RAM, it is temporarily stored on disk (paging / swapping) .

→ **Interrupts** are signals sent to the processor from external hardware devices or internal system components to indicate that they require attention, interrupts pause the CPU's current task to handle urgent hardware-related tasks, enabling efficient multitasking.asynchronous events- they can occur at any time, independent of the processor's current instruction execution.

→ **Exceptions** are unexpected events arising from the currently executing instruction in the processor during execution, synchronous events they occur at specific points in instruction execution.

HDD (Hard Disk Drive) : Uses spinning magnetic platters and mechanical read / write heads, relatively slow due to mechanical movement, typically around 80 - 160 mb/s, often offers the largest storage capacities at the lowest cost per GB.

SDD (Solid State Drive) : uses NAND flash memory with no moving parts, much faster than HDDs, typically up to 500 - 600 MB/s , more resistant to physical shock, silent operation, more expensive per GB then HDDs

NVMe (Non-Volatile Memory Express) : A protocol designed specifically for SSDs conencted via PCIe interface, significantly faster thant SSDs with throughput often exceeding 3, 000 MB's and upt o 7,500 MB/s in latest models

Feature	HDD	SSD (SATA)	NVMe SSD
Technology	Mechanical spinning disks	NAND flash memory	NAND flash + PCIe + NVMe protocol
Speed (Typical)	80-160 MB/s	500-600 MB/s	3,000+ MB/s to 7,500 MB/s
Latency	High	Low	Very low
Durability	Susceptible to mechanical failure	High (no moving parts)	High (no moving parts)
Cost per GB	Lowest	Moderate	Highest

Feature	HDD	SSD (SATA)	NVMe SSD
Use Cases	Bulk storage, backups	General computing	High-performance workloads
Power Consumption	Higher	Lower	Lowest
Form Factor	3.5" or 2.5" drives	2.5" SATA or M.2	M.2, U.2, PCIe

RAID allws balancing between performance, data protection and storage efficiency based on the specific requirements of the use case.

→ RAID 0 offers speed but no redundancy

→ RAID 1 offers full redundance with mirrored disks

→ RAID 5 & RAID 6 offer striping with parity, allowind for fault tolerance with fewer disks used for redundancy.

→ RAID 10 combines mirroring and striping to offer both performance and redundancy but requires more disks.

RAID Level	Storage Technique	Fault Tolerance	Minimum Drives	Use Case / Description
RAID 0	Striping (data split across disks)	None (no redundancy)	2	Improves performance by spreading data over disks, but failure of one disk causes data loss. Used for speed where data integrity is less critical (e.g., video editing).
RAID 1	Mirroring (duplicate copies)	Can tolerate failure of 1 disk	2	Stores identical data on two disks. Provides high fault tolerance, but with 50% storage efficiency. Used for critical systems needing redundancy.

RAID Level	Storage Technique	Fault Tolerance	Minimum Drives	Use Case / Description
RAID 2	Bit-level striping with ECC parity	Can tolerate multiple disk failures	3+	Rarely used today; historical importance. Uses Hamming code for error correction.
RAID 3	Byte-level striping with dedicated parity	Can tolerate failure of 1 disk	3	Used for sequential data processing, less common today.
RAID 4	Block-level striping with dedicated parity	Can tolerate failure of 1 disk	3	Similar to RAID 5 but with dedicated parity disk; not widely used due to bottlenecks.
RAID 5	Block-level striping with distributed parity	Can tolerate failure of 1 disk	3	Balanced performance and fault tolerance. Widely used for general-purpose storage.
RAID 6	Block-level striping with double distributed parity	Can tolerate failure of 2 disks	4	Enhanced fault tolerance with two parity blocks. Used in critical, high-capacity setups.
RAID 10 (1+0)	Mirrored sets in striped array	Can tolerate failure of multiple disks (one per mirrored pair)	4	Combines mirroring and striping for high performance and redundancy. Ideal for databases and critical, high-speed storage.

A **Network Interface Card (NIC)** is a hardware component that enables a device, such as a computer or server to connect to a network. It handles communication

on the physical and data link layers of the network

- Provides a physical interface to connect to wired or wireless networks
- Manages data transmission and reception over the network
- Assigns a unique MAC (Media Access Control) address to the network device.
- Supports communication protocols like Ethernet or Wi-Fi
- Can afford some processing tasks from the host CPU (ex : checksum, calculations)

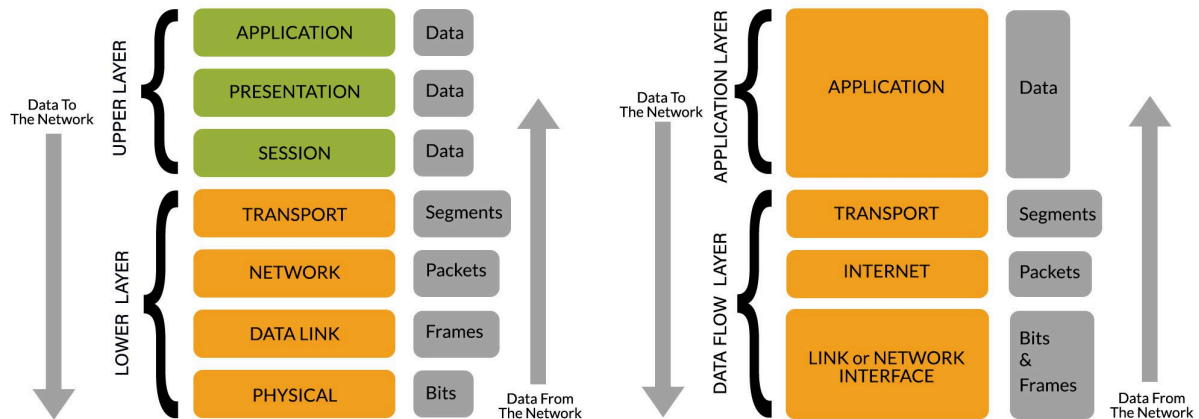
Network Interface Cards are essential for connecting devices to networks, available in various form factors and speed capabilities. Wired NICs are predominant for stable, high-speed connectivity, while wireless NICs provide flexibility and mobility. Advanced NICs (like PCIe and SmartNICs) meet the demands of modern data centers and high performance computing.

A **bare metal server** is a dedicated physical server that is exclusively allocated to a single user or tenant, without any virtualization layer or shared resources. Unlike virtual machines or cloud servers that share hardware among multiple users, a bare metal server provides direct access to the underlying hardware resources such as CPU, RAM, storage and network interfaces.

- Key Characteristics : Exclusive Use, High Performance, Full Control, Enhanced Security, Customization, Stable & Reliable.
 - Bare metal servers can be procured as traditional dedicated servers or integrated into cloud environments where native hypervisors may be used, but still provide direct hardware access without virtualization overhead.
 - Bare metal servers offer the raw, unlayered power of hardware directly to the user, making them ideal for demanding enterprise and data-intensive applications.
-

Networking

OSI MODEL vs TCP/IP MODEL



OSI Model (Open Systems Interconnection)

→ Framework to understand and design interoperable network protocols, vertical layering with strict separation of responsibilities, Protocol-independent, serves as a guideline.

1. Physical Layer : Transmission of raw bits over physical medium
2. Data Link Layer : Node-to-Node data transfer, error detection / correction, convert raw bits into frames
3. Network Layer : Routing and forwarding packets across multiple networks
4. Transport Layer : End-to-End communication, reliability, flow control (ex : TCP, UDP), segmentation
5. Session Layer : Manages sessions or connections between applications
6. Presentation Layer : Data translation, encryption, compression
7. Application Layer : Network Services for end-user applications (ex : HTTP, FTP)

TCP / IP Model (Transmission Control Protocol / Internet Protocol)

→ Practical implementation model underpinning the internet, horizontal layering with some functional overlap, protocol-specific suite that drives the internet

1. Network Interface (Link) Layer : Combines OSI's Physical and Data Link Layers, hardware and media access
2. Internet Layer : Routing of Packets (IP)
3. Transport Layer : End-to-End communication and reliability (TCP, UDP)
4. Application Layer : Combines OSI's Session, Presentation and Application Layers, protocols like HTTP, FTP, SMTP

→ The OSI Model is a detailed 7-layer conceptual guide for standardizing network communication protocols across diverse systems. It helps understand networking concepts in isolation but is less used directly in implementations.

→ The TCP / IP Model is a 4-layer practical suite employed worldwide for real-world network communications, including the internet, combining some OSI functions into single layers for streamlined protocol design and deployment.

→ Both models aim to facilitate data communication across networks but differ in structure, scope and application.

→ In essence, the **OSI model** provides a broad theoretical scope to visualize and plan network communications, while the **TCP/IP model** defines the functional scope of practical protocols used daily in networking and internet communications.

→ IPv6 is designed to overcome the IPv4 limitations, especially the shortage of address space, while enhancing security, auto-configuration and routing efficiency for modern and future internet needs.

IPv4 vs IPv6

Feature	IPv4	IPv6
Address Size	32 bits	128 bits
Address Format	Dotted decimal (e.g., 192.168.1.1)	Hexadecimal (e.g., 2001:0db8:85a3:0000:0000:8a2e:0370:7334)
Address Space	About 4.3 billion addresses	About 340 undecillion addresses
NAT	Required due to address shortage	Not needed because of vast address space
IPSec Support	Optional	Built-in
Autoconfiguration	Manual setup or DHCP	Automatic address configuration
Header Complexity	Complex	Simple
Fragmentation	Handled by routers and sending hosts	Handled by sending hosts only

Subnetting : It is the process of dividing a larger network into smaller, manageable subnetworks (subnets) .

- It improves network performance by reducing broadcast domains
- Enhances security by Isolating segments
- Organizes IP address allocation more efficiently
- The IP address is split into a network part and a host part, the subnet mask defines how many bits are used for the network and how many for the hosts, borrowing bits from the host portion creates multiple subnets within the original network.

CIDR (Classless Inter-Domain Routing) : It is an IP addressing scheme that replaces the traditional class-based IP addressing system by allowing more flexible allocation of IP addresses and routing.

→ Ip address followed by a slash and a number indicating the prefix length of the network portion (192.168.1.0/24)

→ Reduces wastage of IP addresses by allocating address blocks of varying sizes, allows efficient aggregation of routes to reduce routing table size (supernetting) .

→ uses a variable-length subnet mask, allows IP blocks to be divided or aggregated regardless of traditional class boundaries.

→ Subnetting is dividing an IP network into smaller parts, CIDR provides a flexible notation and way to represent subnetted networks.

→ Both use subnet masks or prefix lengths to define network and host portions

→ CIDR enables more precise allocation and routing than traditional subnet classes.

→ Both subnetting and CIDR are essential for efficient IP address management, network segmentation and scaling, especially given the scarcity of IPv4 addresses.

Routing : It is the process of selecting paths in a network along which to send data packets from the source to the destination across multiple networks.

→ Connects different network or subnetworks, enabling data to travel between them

→ Operates at layer 3 (i.e. network layer)

→ Uses IP addresses to make forwarding decisions.

→ Uses routing tables & protocols (OSPF, BGP) to determine the best path for packet delivery, handles traffic between local networks (LANs) and wide area networks (WANs) or the internet, manages traffic flow across complex networks with policies like load balancing and traffic filtering.

Switching : It is the process of moving data frames between devices within the same local network segment (LAN)

→ Connects devices (such as computers, printers) within the same network to enable direct communication.

- Operates primarily at Layer 2 (i.e. Data Link Layer) but can also operate at Layer 3 for multilayer switches.
 - Uses MAC addresses to forward data frames to the correct device
 - Maintains a MAC address table mapping devices to physical ports, forwards data frames only to intended destination ports to reduce network congestion, supports VLANs to logically segment networks withing a single physica switch
 - Routing connects separate networks by selecting optimal paths for data packets, enabling communication over the internet or between different LANs, Switching enables efficient communication within a LAN by forwarding data frames directly between devices based on MAC addresses.
 - Both are essential for network operating, switches create the local network environment, while routers enable inter-network connectivity.
-

DHCP (Dynamic Host Configuration Protocol) : It automatically assigns IP addresses and other network configuration details (like subnet mask, default gateway, DNS server) to devices when they join a network.

- When a device connectes, it broadcasts a DHCP request, the DHCP server responds with an IP address lease along with configuration settings.
- Removes the need for manual IP config, avoids IP conflicts and manages dynamic and static IP assignments.
- Large networks (school, office)

DNS (Domain Name System) : It translates human-friendly domain name into machine friendly IP addresses, enabling users to access websites and services easily.

- When a user types a domain name, the DNS resolver queries DNS servers to find the corresponding IP address
- Simplifies internet navigation by hiding numeric IP addresses, improves network efficiency with caching.
- Everywhere on the internet and private networks where domain names need resolution to IP addresses

NTP (Network Time Protocol) : Synchronizes the clocks of computers and devices over a network to ensure accurate and consistent timekeeping

→ Devices regularly request and adjust their time from a centralized NTP server or hierarchical time sources

→ Essential for time-sensitive transactions, logging, security protocols, and network coordination

MAC Addressing (Media Access Control) : A unique identifier assigned to a network interface card (NIC) at the hardware level.

→ 48 bit address usually represented as six groups of two hexadecimal digits separated by colon or hyphens (00:1A:2B:3C:4D:5E)

→ Used to identify devices uniquely on a local network at the Data link layer

→ Essential for local communication and switching within the same network segment

ARP (Address Resolution Protocol) : It is a network protocol used to map the IP address of a device to its MAC address

→ used to help devices on a local network find the hardware (MAC) address associated with a given IP address, necessary for data link layer communication.

→ when a device wants to send data to an IP address on the same local network, it needs the corresponding MAC address, it sends the ARP request and the matching IP replies with its MAC address in an ARP reply, the sender caches this mapping in its ARP table for future use.

→ ARP is essential for network communication on local networks, enabling devices to translate IP addresses to MAC addresses, thus allowing data to be sent across the physical network, without ARP, devices would not know how to physically locate each other despite having the IP information.

NAT (Network Address Translation) : It is a technique to modify the IP address information in IP packet headers while they are in transit across a router or firewall. It enables multiple devices on a local private network to share a single public IP address for accessing the internet

→ Conserves the limited number of public IPv4 addresses by allowing private IP addresses internally, enhancing security by hiding internal IP addresses from external networks, enables multiple devices within a private network to access external networks through one public IP.

→ Maintains a NAT translation table to map each internal IP and port to the public translated IP and port

Port Forwarding : It is a NAT technique used to allow external devices to access services on a private network by mapping a specific external port to an internal IP address and port

→ Incoming traffic to a specific public IP address and port is forwarded to a designated private IP and port inside the network. The router/firewall listens on the configured port and redirects the traffic accordingly.

→ NAT and Port forwarding together enable effective use of scarce IP addresses while allowing controlled external access to internal network resources.

A **Firewall** is a network security device (hardware or software) that monitors and controls incoming and outgoing network traffic based on predetermined security rules. It acts as a barrier between a trusted internal network and untrusted external networks (like the internet) .

→ Inspects every packet entering or leaving the network, uses rules based on IP addresses, port numbers, and protocols to allow or block the traffic

→ Filters traffic to protect against unauthorized access, malware and attacks like denial-of-service (Dos).

→ Maintains logs and can alert administrators about suspicious activity.

Protocol	Purpose & Use	Port (Default)
HTTP (Hypertext Transfer Protocol)	Used for transferring web pages over the internet. Non-secure traffic.	80
HTTPS (HTTP Secure)	Secure version of HTTP using encryption (SSL/TLS) to protect data.	443
FTP (File Transfer Protocol)	Transfers files between client and server; supports multiple modes.	21
SSH (Secure Shell)	Provides encrypted remote login and command execution on networked machines.	22
SNMP (Simple Network Management Protocol)	Used for monitoring and managing network devices remotely.	161 (UDP)

Hub : A simple device that connects multiple ethernet devices, making them act as a single network segment, operates at physical layer, broadcasts incoming data to all connected ports, no filtering or routing of data

Switch : connects devices in a network and filters data based on MAC addresses, operates at data link layer, sends data only to the intended recipient port, reducing collisions and improving efficiency, full duplex communication.

Bridge : Connects two or more LAN segments to reduce traffic, operates at data link layer, filters data by MAC addresses, forwarding only necessary packets.

Repeater : regenerates and amplifies weak network signals to extend the transmission devices, operates at physical layer, does not perform filtering or routing

Router : connects multiple networks and routes data packets based on IP addresses, operates at network layer, enables communication between different network architectures (LAN, WAN)

Gateway : Interface between two different network protocols or architectures, can work at multiple OSI layers, performs protocol conversion, data translation and format conversion

Modem : Modulates and demodulates digital data over analog communication lines, converts digital signals to analog for transmission and vice-versa.

Access Point : wireless device that allows wireless devices to connect to a wired network, provides wi-fi connectivity and manages wireless traffic, operates at data link layer

ARQ (Automatic Repeat reQuest) is an error-control protocol used for reliable data transmission over noisy or unreliable communication channels. It uses acknowledgements and timeouts to detect errors and trigger retransmissions of lost or corrupted data.

→ Stop & wait ARQ, go back n ARQ, Selective repeat ARQ

→ ARQ provides guaranteed reliable data transmission by detecting errors and managing automatic retransmissions. Its variants balance simplicity and efficiency based on network conditions and application needs.

→ **MAC (Media Access Control)** Address is a unique hardware identifier assigned to network interfaces for communication at the data link layer, it is a 48-bit address usually written in hexadecimal.

→ **VLANs (Virtual Local Area Networks)** is a logical segmentation of a physical LAN into multiple distinct broadcast domains. Devices on different VLANs behave as if they are on a separate networks even if physically connected to the same switch. These improves security, reduce broadcast traffic, and help with network management.

→ VLANs allow multiple virtual networks within the same physical infrastructure, improving security and efficiency.

Switching Type	Description	Use Case/Characteristics
Circuit Switching	Establishes a dedicated communication path between two devices for the duration of the session.	Used in traditional telephone networks; guarantees bandwidth.
Packet Switching	Data is divided into packets and sent independently over the network; packets may take different routes.	Used in the Internet; efficient and robust; may experience delay.
Message Switching	Entire message is sent to each intermediate node and stored until the next node is ready to receive it.	Older method; requires large buffer space; slower than packet switching.

→ Supernetting is the process of combining multiple smaller networks into a larger one called a supernet, primarily to simplify routing by reducing the number of entries in routing tables, often used by ISPs.

RARP (Reverse Address Resolution Protocol) : It is used by a device to request its IP address from a network server using its known physical MAC address. It translates a MAC address into an IP address

ICMP (Internet Control Message Protocol) : ICMP is used for network diagnostics and error-reporting functions, reports errors, sends control messages and manages traffic in IP networks, ex : PING uses ICMP echo request and reply to test reachability of a host.

IGMP (Internet Group Management Protocol) : It manages group memberships for IP multicast, allows hosts and routers to establish multicast group memberships dynamically. Efficient delivery of multicast packets (one-to-many communication).

Distance Vector Routing : Each router maintains a routing table (distance vector) with the best known distance to every destination and the next hop to reach it.

→ Routers periodically share their distance vectors with directly connected neighbors, upon receiving neighbors distance vectors, routers update their tables based on the bellman-ford algorithm minimizing path cost,

Link State Routing : Each router has complete knowledge of the entire network topology.

→ Routers broadcast link state advertisements (LSAs) to all routers, every router builds a complete map of the network, uses dijkstra's algorithm to compute the shortest path to every destination.

Feature	Distance Vector	Link State
Knowledge of Network	Limited to neighbors	Complete network topology
Algorithm	Bellman-Ford	Dijkstra's shortest path
Information Exchange	Distance vectors between neighbors	Link state advertisements to all routers
Convergence	Slower, loop-prone	Faster, loop-free
Resource Usage	Less CPU and memory	More CPU and memory

Feature	Distance Vector	Link State
Scalability	Limited (small to medium networks)	Scalable for large networks

→ RIP relies on periodic broadcast updates and has limitations in terms of hop count and network size.

→ OSPF builds a complete network topology and calculates shortest paths, supporting hierarchical networks via areas

→ BGP is essential for internet routing, exchanging path information between ISPs and large networks.

→ EIGRP combines distance vector and link state features, offering efficient, fast and flexible routing primarily in Cisco environments

Protocol	Type	Algorithm	Use Case	Characteristics
RIP (Routing Information Protocol)	Distance Vector	Bellman-Ford	Small to medium-sized networks	Simple, uses hop count as metric, max hop count 15, slow convergence, limited scalability
OSPF (Open Shortest Path First)	Link State	Dijkstra's SPF	Large enterprise networks	Fast convergence, scalable with areas, supports VLSM and CIDR, complex setup
BGP (Border Gateway Protocol)	Path Vector (EGP)	Path Vector	Internet-wide routing	Manages routing between autonomous systems (AS), policy-based routing, slow convergence, highly scalable

Protocol	Type	Algorithm	Use Case	Characteristics
EIGRP (Enhanced Interior Gateway Routing Protocol)	Hybrid (Cisco proprietary)	Diffusing Update Algorithm (DUAL)	Enterprise networks, Cisco-centric	Fast convergence, supports multiple metrics (bandwidth, delay, load), supports unequal cost load balancing, requires Cisco equipment mostly

→ Routing is the path determination process where routers learn information about the network topology and compute the best routes.

→ Forwarding is the actual movement of packets based on routing tables, sending them from input to output interfaces.

Aspect	Forwarding	Routing
Definition	The process of moving packets from an input interface to the appropriate output interface within a router or switch.	The process of determining the optimal path for packets from source to destination through the network.
Purpose	Transmit packets quickly through the device based on pre-established routing information.	Build and maintain routing tables that provide those paths.
Operation Level	Performed at each router as packets arrive (Data Plane).	Performed periodically or dynamically by routers to update forwarding tables (Control Plane).

→ Packet Fragmentation is the process of breaking a large packet into smaller manageable pieces called fragments so that they can be transmitted over a network that has a smaller maximum transmission unit (MTU) than the original packet size.

→ QOS (Quality of Service) parameters define the performance characteristics of a network service, aimed at ensuring predictable and reliable delivery of data : latency, Jitter (Variation in packet delay over time), Throughput (The rate at which data is successfully transmitted over the network, usually measured in bits per second bps)

→ Congestion occurs in a computer network when the traffic load exceeds the network's capacity to handle it effectively. This leads to network performance degradation similar to traffic congestion on a busy road.

- **Cause:** Too many packets are sent simultaneously, overwhelming network nodes or links.
- **Effects:** Increased latency (delay), packet loss, jitter, slower data transmission, and reduced throughput.

→ Network congestion control is essential to maintain efficient, reliable, and fair operation of modern networks, ensuring smooth data flow and a better user experience.

→ A Port is a logical numeric identifier for specific processes or network services on a device (0 - 65535). Ports allow multiple applications on a single device to use network communication simultaneously without interference.

- reserved ports : 0 - 1023

→ **Ports** identify and differentiate multiple services running on the same machine.

→ A Socket is a combination of an IP address and a port number, representing one endpoint of a two way communication link between two devices over a network. It defines the connection interface for processes to send / receive data.

→ **Sockets** represent active network communication endpoints between two devices, combining IP address and port.

Feature	FTP (File Transfer Protocol)	TFTP (Trivial File Transfer Protocol)
Full form	File Transfer Protocol	Trivial File Transfer Protocol
Protocol type	Connection-oriented (TCP)	Connectionless (UDP)

Feature	FTP (File Transfer Protocol)	TFTP (Trivial File Transfer Protocol)
Ports	Control on port 21, Data transfer on port 20	Port 69
Complexity	More complex with authentication and commands	Simple with minimal commands
Authentication	Provides user authentication	No user authentication
Transfer modes	Supports ASCII and binary	Only binary
Error handling	Supports error recovery	Limited error handling
Transfer control	Supports resuming interrupted transfers	No support for resuming
Usage	General-purpose file transfer over Internet	Lightweight file transfers, used in booting routers or network devices configuration
Security	No encryption by default but can use FTP over TLS	No encryption

Protocol	Purpose	Description
SMTP	Sending email	Simple Mail Transfer Protocol; used for sending outgoing mail from client to server or between servers.
POP3	Receiving email	Post Office Protocol version 3; used to download emails from server to local device, typically deleting emails from server afterward.
IMAP	Receiving and managing email	Internet Message Access Protocol; allows email synchronization with server, supports multiple devices accessing same mailbox.

Protocol	Purpose	Description
Telnet	Remote login to another computer	Sends all data in plaintext; insecure.
SSH	Secure remote login	Encrypts all data; more secure; supports tunneling and authentication.

Operating System

→ An Operating System is a software that manages computer hardware and software resources and acts as an interface between the computer user and the hardware. Its main function include managing hardware, running applications, providing user interfaces, handling input / output operations, managing files and memory, ensuring security and facilitating communication between different software and hardware components.

- Batch OS : Processes batches of jobs with minimal user interaction, jobs are collected, grouped , and processed sequentially
- Multiprogramming : Allows multiple programs to reside in memory at once. The CPU switches between them to utilize resources efficiently, reducing idle time
- Multiprocessing : Uses two or more processors (CPUs) within one system, increasing speed and reliability through parallel processing.
- Time-Sharing : Enables multiple users to use a computer system interactively and concurrently by rapidly switching between tasks, giving each user time slices of CPU
- Real Time Os : Responds to input or events within a strict time limit.
- Distributed Os : Manages multiple independent computers connected over a network.
- Mobile Os : Designed for smartphones

→ The Kernel is the central core of an operating system. It acts as a bridge between software applications and hardware, managing resources like CPU, memory and devices. The kernel handles process management, memory management, device management and system calls ensuring smooth operation and resource allocation for applications.

- Monolithic : All os and device drivers run in kernel space, offers speed and direct communication
- Microkernel : Only essential core functions run in kernel space, other services operate in user space
- Hybrid : Combines features of monolithic and microkernel
- ExoKernel : exposes hardware resources directly to applications, allowing them to manage resources with little abstraction.

- Nano Kernel : Provides only the minimal necessary code for low-level hardware abstractions, almost all services handled elsewhere.



In an operating system, the CPU operates in two distinct modes : User mode(1 → unprevilged mode) and Kernel Mode (1 → priviledged mode). These modes control the level of privilege and access to system resources.

→ User mode is a restricted mode running application software with limited access to protect the system from faults or malicious actions

→ Kernel mode has unrestricted access, allowing it to directly manage hardware and critical system functions, but also means errors here have more serious consequences.

→ These separations enhances system security, stability and fault isolation by isolating user programs from direct hardware and critical resource access.

→ System calls are special functions provided by the operating system that allow user-level applications to request services from the OS kernel. They act as an interface between a running process (in user mode) and the operating system, enabling processes to perform tasks that require kernel-level privileges or hardware access. Essentially system calls allows programs to switch from user mode to kernel mode safely to execute these privileged operations.

Type	Description	Examples of Operations
Process Control	Manage processes: creation, termination, execution control, and synchronization.	fork(), exec(), exit(), wait()
File Management	Perform operations on files like opening, reading, writing, closing, and deleting files.	open(), read(), write(), close(), delete()
Device Management	Control and manage device request, release, and interaction with hardware devices.	ioctl(), request_device(), release_device()
Information Maintenance	Retrieve or modify system or process information such as system time, process ID, and system status.	getpid(), settimeofday(), uname()

Type	Description	Examples of Operations
Communication	Facilitate communication between processes through message passing, synchronization, and shared memory.	pipe(), shmget(), send(), receive()

Process : A process is a program in execution, it is an active entity that represents the execution of a program along with its current activity, such as the program counter, registers and variables. Multiple processes can be associated with a single program running multiple times.

Job : A job is a unit of work or a program submitted to the system for execution. It may consist of one or more processes and is generally a higher level concept used in batch os.

Task : A task can be considered a unit of work or a smaller operation within a process. It is often used interchangeably with a process but sometimes refers to threads or parts of a process.

→ Processes transition through states like new, ready, running, waiting, and terminated, known as the **process lifecycle**.

Process State	Description
New	The process is being created.
Ready	The process is loaded in memory and waiting to be assigned to the CPU for execution.
Running	The process is currently being executed on the CPU.
Waiting (Blocked)	The process is waiting for some event to occur (e.g., I/O completion).
Terminated	The process has finished execution and is being removed from the system.

→ A Process control block (PCB) is a data structure used by the operating system to store all the information about a specific process.. It acts as the repository for any data the os needs to manage and control the process during its lifecycle.

- Process Id (PID) → Unique Identifier for the process
- Process State → Current state like ready, running, terminated etc

- Program Counter → Address of the next instructions to execute
- CPU registers → Contents of CPU registers when the process is not running
- Memory Management Information → Details about memory allocated to the process, such as base and limit registers, page tables
- Scheduling information → Priority, pointers to scheduling queues
- List of open files → files currently opened by the process
- Accounting information → CPU usage, time limits, etc.



The PCB is essential for context switching as it allows saving the state of a process when switching it out and loading it back when scheduled again. The OS maintains a process table containing PCBs for all active processes.

→ A Thread Control Block (TCB) is similar to a PCB but stores the information about a thread, which is the smallest unit of execution within a process.

- Thread Id → unique identifier for the thread
- Thread state → running, ready, or blocked
- **Program Counter:** Address of the next instruction the thread will execute.
- **CPU Registers:** Registers for the thread's execution state.
- **Stack Pointer:** Points to the thread's stack in memory.
- **Scheduling Information:** Thread priority and other scheduling data.
- **Thread-Specific Data:** Any thread-local storage.

→ Since threads share the same resources (address space and open files) of their parent process, the TCB mainly holds data specific to thread execution and scheduling.

Aspect	Job	Task	Process	Thread
Definition	A broad unit of work or set of	A single unit of work or	A program in execution with	Smallest unit of execution

Aspect	Job	Task	Process	Thread
	activities to be done, may contain multiple tasks or processes.	operation within a job.	system resource allocation.	within a process sharing its resources.
Scope	High-level, may span many tasks or processes.	Specific part of a job or process.	Allocated CPU, memory, I/O resources for execution.	Shares memory and resources of parent process.
Resource Use	Managed at job level for multiple tasks/processes.	Managed within job or process.	Has its own memory and resources.	Uses shared resources of process, lightweight.
Control	Executed in batch or scheduled systems.	Smaller segment of job workload.	Controlled by OS with process control block.	Controlled by thread control block, lighter weight.
Execution	Can last longer, made up of multiple processes/tasks.	Specific operation unit.	Runs independently, heavier to create/terminate.	Runs concurrently with other threads in same process.

Term	Description	Examples / Context
Multiprocessing	Using two or more CPUs/processors simultaneously to execute multiple processes independently.	Servers, high-performance computing.
Multithreading	Multiple threads within a single process run concurrently, sharing memory space.	Web browsers, video games, GUI applications.
Multitasking	The OS handles multiple processes by rapidly switching between them to give the illusion of simultaneous execution.	Running multiple applications on a PC.

→ Context switching is the process in an operating system where the CPU switches from executing one process or thread to another. This involves saving the state (context) of the currently running process or thread and loading the saved state of the next one to be executed.

→ It enables multitasking by rapidly switching between processes or threads giving the illusion of simultaneous execution.

What is saved in context ?

- CPU registers (general purpose, program counter, stack pointer)
- Memory management info (virtual to physical address mapping)
- Process or thread stack (function call history)

→ Operating systems use different schedulers to manage processes at various stages of their lifecycle. These schedulers handle process selection and movement between different states or memory levels.

→ The Long-term scheduler controls which jobs enter the system and maintains a good mix of I/O bound and CPU bound processes to optimize system performance

→ The Medium-term scheduler manages the swapping of processes in and out of main memory to balance memory usage and keep the CPU busy.

→ The Short-term scheduler is the most active scheduler that assigns CPU to processes ready to execute, ensuring efficient use of the CPU through fast scheduling and context switching.

Scheduler Type	Description	Key Points
Long-term Scheduler (Job Scheduler)	Selects processes from the job pool (usually secondary storage) and loads them into main memory into a ready state. Controls the degree of multiprogramming (number of processes in memory).	- Controls job admission to the system.- Balances CPU-bound and I/O-bound processes.- Runs less frequently.- May be absent in some time-sharing systems.
Medium-term Scheduler	Temporarily removes (swaps out) processes from main memory to secondary storage and brings them back (swaps in) to manage memory and degree of multiprogramming. Handles suspended processes waiting for I/O or resource availability.	- Manages swapping of processes to optimize memory use.- Controls process suspension and resumption.- Executes more frequently than long-term scheduler but less than short-term.
Short-term Scheduler (CPU Scheduler)	Selects one process from the ready queue in main memory for CPU execution. Performs frequent process switching to optimize CPU utilization.	- Runs very frequently.- Responsible for actual CPU assignment.- Uses different scheduling algorithms.- Also called the dispatcher.-

Scheduler Type	Description	Key Points
		Handles context switching time.

Algorithm	Type	Preemptive	Advantage	Disadvantage
FCFS	Non-preemptive	No	Simple and fair	Long average wait, convoy effect
SJF	Both	Optional	Minimizes average waiting time	Starvation of long jobs
Round Robin (RR)	Preemptive	Yes	Fair, good for time-sharing	High context-switch overhead
Priority Scheduling	Both	Optional	Prioritizes important processes	Starvation possibility
Multilevel Queue	Both	Usually No	Good for different job types	No queue mobility
Multilevel Feedback Queue	Both	Yes	Flexible, avoids starvation	Complex implementation

Multilevel Queue Scheduling : Processes are divided into multiple queues based on priority or type , each queue has its own scheduling algorithm, processes don't move between queues

Multilevel Feedback Queue Scheduling : Similar to multilevel queue but allows processes to move between queues based on their behaviour and ageing, processes that use too much CPU are moved to lower priority queues, designed to optimize response time and avoid starvation



Process synchronization is a mechanism in operating systems to coordinate the execution of multiple processes such that they can safely share resources and access shared data without interfering with each other

→ A race condition occurs when multiple processes or threads access and manipulate shared data concurrently, and the final outcome depends on the timing or sequence of the execution. This causes unpredictable results because the processes "race" to read or write the data.

→ It happens mostly in critical section where processes access shared resources

→ The outcome depends on which process finishes first, leading to possible data inconsistencies.

→ The critical section problem involves ensuring that multiple processes do not simultaneously execute their critical sections i.e. the parts of the programs accessing shared data.

→ Conditions to satisfy : mutual exclusion, progress, bounded waiting (each process should have a fair chance to enter the critical section without starvation

How does process synchronization work ?

1. Entry Section : Code before entering the critical section, decides if a process can enter
2. Critical Section : Contains code where shared data is accessed, only one process allowed
3. Exit Section : Code that permits other waiting processes to enter after the current process leaves
4. Remainder Section : Rest of the process code, outside critical section.

Peterson's solution is a classical software algorithm used to solve the critical section problem for two processes. It ensures mutual exclusion, guaranteeing that two processes do not enter their critical sections simultaneously thereby preventing race condition.

→ It uses two shared variables between the two processes :

- flag : a boolean array where each element indicates whether a process wants to enter the critical section
- turn : an integer variable indicating which process's turn it is to enter the critical section

→ A semaphore is a synchronization tool represented by an integer variable, it controls access to shared resources by multiple processes through two atomic operations :

- P (wait) : decreases the semaphore value, if the value is zero, the process waits
- V (signal) : Increases the semaphore value, if there are waiting processes, one is allowed to proceed.

Primitive	Description	Use Case / Notes
Semaphore	A variable or abstract data type used to control access to a common resource by multiple processes.	Can be binary (0 or 1) for mutual exclusion or counting semaphore for resource pools. Uses P() (wait) and V() (signal) operations.
Mutex (Mutual Exclusion)	A lock that allows only one thread/process to access a resource at a time.	Binary locking mechanism aimed at protecting shared resources. Unlike semaphores, only the owning thread can release the mutex.
Monitor	A high-level synchronization construct that combines mutual exclusion with condition variables to manage safe access to shared data.	Encapsulates data, operations, and synchronization in one module. Used in many modern programming languages.
Spinlock	A lock where the thread spins in a loop waiting for the lock to become available instead of sleeping.	Efficient for short wait times but wastes CPU cycles. Often used in low-level or kernel synchronization.
Condition Variable	Allows threads to wait for certain conditions within the critical section to be met before continuing execution.	Used for signaling between threads, often in conjunction with mutexes or monitors.



A state is **safe** if there exists a sequence of all processes where each process can get all the resources it needs and finish without causing deadlock.

→ A Deadlock occurs when a set of processes in a system get stuck, each waiting for resources that are held by other processes in the set, causing all of them to be blocked indefinitely.

→ Necessary conditions : Mutual Exclusion, Hold & Wait, No Preemption, Circular Wait

→ Deadlock prevention ensures that at least one of the necessary conditions is never allowed

→ Deadlock Avoidance algorithms dynamically analyze resource allocation to ensure that the system never enters an unsafe state where deadlock is possible

- Banker's Algorithm : Simulates resource allocation to check if granting a request keeps the system in a safe state.

→ In deadlock detection, system allows deadlocks but periodically checks for their occurrence, uses resource allocation graphs or wait-for graphs to detect cycles, once detected, deadlock recovery is initiated.

→ In Deadlock recovery, process termination (abort one or more process), resource preemption, rollback (roll processes back to safe states if supported

Aspect	Starvation	Deadlock	Livelock
Definition	When a process waits indefinitely because other higher-priority processes continuously get resources first.	When processes get stuck waiting for each other to release resources, causing a circular wait.	When processes continuously change their states to avoid conflict but neither makes progress.

→ Virtual Memory is a memory management technique used by the operating systems to give applications the illusion of a large and continuous block of memory, even if the physical RAM is limited, it allows a system to run larger programs or multiple programs simultaneously by using both RAM and a portion of secondary storage to extend physical memory.

→ Demand paging is a technique used in virtual memory where pages (fixed sized blocks of memory) are loaded from disk into RAM only when they are needed (i.e on demand). Instead of loading an entire process into memory, only the necessary pages are brought in, this reduces the memory requirements and allows more processes to fit into the available RAM, when a page is needed but not in memeory, a page fault occurs.

→ A Page fault happens when a process tries to access a page that is not currently loaded into physical memory

→ Page faults are normal in demand paging but too many page faults can degrade performance (called thrashing).

→ Thrashing is a condition in virtual memory systems where the operating system spends the majority of its time swapping pages between physical memory (RAM) and seconday storage (disk) instead of executing actual processes. This causes severe performance degradation and system slowdown.

→ The working set model is a concep used to reduce thrashing by tracking the set of pages a process is actively using during a given time window.



Fragmentation is an issue that occurs in memory management when free memory is broken into small pieces over time due to loading and unloading processes. It leads to inefficient use of memory and can prevent allocation of memory to processes even if enough total free space exists.

Internal Fragmentation : Happens when a fixed size memory block is allocated to a process, but the process requires less space than the block size.

External Fragmentation : Occurs when free memory is available but is scattered in small non-contiguous blocks.

→ Compaction is a memory management technique used to eliminate external fragmentation by rearranging the contents of memory to place all free memory

together in one large contiguous block. Compaction is a **time-consuming process** because it involves copying process data to new memory locations.

Aspect	Description
What is it?	Rearranging occupied memory blocks to create larger contiguous free memory.
Purpose	To solve external fragmentation by compacting free memory.
How it works	Moves all processes to one side, free space to the other.
Requirements	Dynamic relocation support for process addresses.
Downsides	CPU idle time, high overhead during rearrangement.

→ The Buddy system is a memory allocation and management technique designed to efficiently allocate memory in power-of-two sized blocks. It helps reduce fragmentation and speeds up allocation and deallocation of memory.

Interprocess Communication (IPC) allows multiple processes to communicate and synchronize their actions, which is essential for cooperation and data sharing among processes.

- Pipes : Pipes are simple communication channels used for data exchange between related processes., generally unidirectional (data flows in one direction) .
- Message Queues : It allow processes to send and receive messages asynchronously, messages are stored in a queue in the kernel until retrieved by the receiving process.
- Shared Memory : It allows multiple processes to access a common memory region, fastest form of IPC as data is directly read and written in a shared address space.
- Sockets : Sockets provide bidirectional communication, often over a network, can be used between processes on the same machine or different machine.
- Signals : These are limited messages sent to processes to notify events or interrupts, used for simple interprocess communication, like notifying a process to stop, resume or handle an event, not used for data transmission.

Essentials

→ Nutanix's hyperconverged infrastructure implements many of the critical functions of a distributed operating system : abstracting hardware, distributing services and resources, managing failures and enabling simplified, unified management across all physical nodes. In essence, Nutanix makes the entire cluster behave like a single, distributed, highly available system which is the central goal of any distributed OS.

→ Microkernels are considered more secure because they run only minimal core functions in kernel space, isolating most OS services in user space, which confines potential damage from bugs or malicious code, but extra overhead is added due to frequent IPC and context switching between user space and kernel space.

→ `fork()` creates a new child process by copying the current process, but `exec()` replaces the current process's program with a new program.

→ The convoy effect in operating systems is a situation where a few slow processes (usually cpu intensive) block other fast processes, causing the entire system's performance to slow down. In simpler terms, it's like a convoy of vehicles moving at the speed of the slowest vehicle, causing delays for all others behind it. To prevent the convoy effect, OSes use preemptive scheduling algorithms like Round Robin, which prevent any single slow process from blocking others too long.

→ Multithreading is more resource-efficient, faster for context switching and enables easier data sharing within a single application compared to multiprocessing.

→ Peterson's algorithm is not used in modern multiprocessors because it relies on busy waiting, which wastes CPU resources and assumes strict memory operation ordering that modern CPUs do not guarantee due to optimizations like out-of-order execution and caching. Also modern processors provide better atomic processors for synchronization, making Peterson's algorithm impractical.

→ In a mesh topology, every node is directly connected to every other node. This means that if one connection or even multiple connections fail, data can still be routed through one or more alternate paths. The redundancy in connections ensures the network's operation remains unaffected by most individual link failures, making mesh topology extremely fault-tolerant compared to other topologies like star, ring or bus.

→ Data centers prefer optical cables because they offer high bandwidth, low latency and long distance data transmission without significant loss. They are immune to electromagnetic interference, more energy-efficient and scalable for future data demands, making them ideal for reliable and fast data center connectivity.

→ Linux is a modern, open-source and widely used unix-like system, unix is older , proprietary and less flexible

Directories	Usage
/bin	essential binaries (cp, mv , ls)
/etc	configuration files
/home	user directories
/var	variable data (logs, mail)
/tmp	temporary files
/usr	user applications & libraries
/dev	device files
/proc	process & kernel info

→ Linux is preferred for servers due to a combination of technical and economic advantages : cost-effectiveness, stability & reliability, security, performance & efficiency, scalability & flebility, cusomtizability, strong community and support.

Read (r)	View file contents (value : 4)
Write (w)	Modify file contents (value : 2)
Execute (x)	Run the file (value : 1)

Symbolic : `chmod u+rx, g+rx, o+rx filename`

Shortcut : `chmod a+rx filename`

Numeric : `chmod 777 filename`

`chown user file` → change owner

`chgrp group file` → change group

Owner : The individual who created the file (can be changed by admin)

Group : The file can be assigned to a group for collaborative access

Others : All other users not covered by owner / group

→ Use grep for simple searching / filtering of patterns, use awk for advanced data extraction, manipulation and reporting on structured text.

ps aux	snapshot of process
top	dynamic real time process monitor
&	run in background
jobs	lists background jobs
fg % 1	brings job to foreground
bg %1	continue job in background
df -h	disk usage
du -sh dir/	size of directory
free -m	memory usage
uptime	system uptime & load average
vmstat	CPU, memory, I/O stats
dmseg	kernel messages

apt update (Debian / ubuntu)	refresh repositories
apt install package (Debian / ubuntu)	install software
apt remove package (Debian / ubuntu)	remove software
yum install package	RedHat / CentOS
dnf remove package	RedHat / CentOS



Package managers save time, reduce complexity, improve security and maintain system integrity for Linux users and administrators.

Foreground Processes : These run directly attached to the terminal. When you start a process in the foreground, the shell waits for it to finish before you can use the terminal again. You can interact with the process and it can receive input from the keyboard. You can stop it with Ctrl + C.

Background Processes : These run independently of the terminal and allow you to keep using the shell while they execute. You start a background process by appending & at the end of the command (ex : sleep 30 &). Background

processes don't receive keyboard input and don't block the terminal. You can list them using jobs and bring them back to the foreground with fg if needed.



Shell : Command interpreter that provides an interface between user and OS

Shell Scripting : Writing scripts (.sh files) to automate repetitive tasks.

Shell variables : defined and used only within the current shell session, not inherited by child processes or subshells, typically used for temporary, local storage of values. They vanish when the shell session ends. (ex : VAR = hello)

Environment variables : Exported from the shell and inherited by child processes (like scripts or programs launched from that shell) , used to pass configuration or system-wide settings to processes, created by exporting a shell variable (export VAR), ex : PATH, HOME, SHELL. persistent environment variables can be set in shell config files like .bashrc or .profile.

→ This distinction helps maintain process isolation and control configuration propagation in Unix like systems.

adduser anshul → add new user

passwd anshul → set passwd

deluser anshul → delete user

groupadd devs → add group

usermod -aG devs anshul → add anshul to devs

File : /etc/sudoers (edit with visudo)

→ The visudo command is preferred over directly editing the sudoers file (/ etc/ sudoers) because it ensures safe, atomic and error-checked modification of sudo privileges, making it the best practice tool for sudoers editing.

systemd	Modern init system for linux, manages services and daemons
syslog (file)	general system logs
auth.log (file)	login attempts, sudo usage

dmseg (file)	kernel messages
----------------	-----------------

`journalctl -u service-name` → to check why a service failed to start

<code>ip addr</code>	shows interfaces
<code>ping</code>	check connectivity
<code>netstat -tulnp / ss-tulnp</code>	check listening ports
<code>ssh-keygen</code>	generate key pair



SSH's encryption and robust authentication make it the secure and modern choice for remote access, replacing the insecure Telnet Protocol.

→ A daemon is a background, non-interactive process that runs independently of user control, typically starting at boot and running throughout system operation. Examples : sshd, httpd, cron. Daemons generally don't interact with the user directly.

→ A service is broader, it's a program that performs a specific function, which can be managed (started, stopped, restarted) by service managers like systemd. Most services are provided by daemons, but not always and the term "service" is commonly used in modern Linux to refer to something managed by the init system (systemd, upstart, etc).

→ In summary, daemon is the long running background process itself, service is the higher level function, often managed via system tools, which may run as a daemon.

CapEx (Capital Expenditure) : Refers to major, long term investments a company make in physical or intangible assets, these expenditures are used over a long period (more than one year) . It is capitalized on the balance sheet and depreciated or amortized over the asset's useful life.

OpEx (Operating Expenditure) : Refers to the day-to-day expenses needed to run the business, these costs benefit the company within a short timeframe (

usually less than a year). They are often recurring, smaller expenses necessary for daily operations.

→ In essence, **CapEx** invests in future growth and capacity, whereas **OpEx** covers operational efficiency and day-to-day business continuity. Both are essential but impact financial statements differently and are managed separately for accounting and tax purposes.

Block Storage : Data is divided into fixed size blocks (chunks) and each block is stored separately with a unique address. The operating system or storage controller manages these blocks, which can be independently accessed and modified. It works like a raw hard drive or SSD and is used for databases, virtual machines and applications needing fast read / write and low latency. It generally offers high performance and quick data transfer.

Object Storage : Data is stored as discrete objects, each containing the data itself, metadata and a unique identifier, uses a flat namespace, no hierarchical file system like traditional storage. Suitable for storing large amounts of unstructured data such as multimedia files, backups, and big data. highly scalable with near-infinite storage capacity.

→ A Virtual Private Cloud (VPC) is a secure, isolated virtual network environment within a public cloud infrastructure. It allows organizations to run resources like servers, databases and applications in a logically separate and controlled space, providing the benefits of a private cloud while leveraging the scalability of the public cloud.

→ Thinking of a VPC like having your own private network space inside a larger shared cloud environment, where you can customize security, connectivity and resource allocation as if you had your own data center but without the hardware costs. This makes VPC ideal for running scalable, flexible and secure cloud-based applications.

Scalability : Refers to the ability of a system to handle increasing workloads by adding resources, either by scaling up (vertical scaling) or scaling out (horizontal scaling). It is typically a planned, long-term process to accommodate growth, involves adding resources permanently or semi-permanently to meet future demands, requires manual or automated intervention but is often part of capacity planning.

Elasticity : Refers to the ability to automatically and dynamically allocate or release resources in response to real-time changes in workload demand. It is an automatic, short-term adjustments to handle instantaneous spikes or drops,

resources can be scaled up or down quickly based on current needs, often configured with pre-set rules.



User Serverless when handling unpredictable, bursty, event-based or infrequent workloads, want rapid development and minimal infrastructure management, application logic is naturally stateless and can run within platform-imposed time limits.

Choose containers if you need complete control over the environment, stateful or long-running processes or have complex dependencies and compliance requirements.

Migration Strategies (6 "Rs") :

Rehost (Lift & Shift)	Move applications to cloud with minimal changes
Replatform	Make small optimizations
Repurchase	Switch to SaaS alternatives
Refactor (Re-architect)	Redesign app for cloud-native
Retire	Decommission unused apps
Retain	Keep some workloads on-premises

RPO (Recovery Point Objective) : Maximum acceptable data loss (data loss tolerance)

RTO (Recovery Time Objective) : Maximum acceptable downtime (downtime tolerance)



Kubernetes (K8s) Open Source container orchestration system

Pod	Smallest Deployable unit
Node	Worker machine running pods
Cluster	Collection of nodes managed by control plane

Deployment	Defines desired state for pods
Service	Provides networking & load balancing

Virtual Machine Lifecycle Management

1. Provisioning - Create VM with defined CPU, memory, storage, network
2. Configuration - Install OS, drivers, app,s
3. Operation - Regular usage by workloads
4. Monitoring - Track performance, resource usage
5. Maintenance - Apply updates, patches,
6. Decommissioning - Archive or delete unused VMs.

→ Ballooning is a memory management technique used in virtualized environments to reclaim unused memory from a virtual memory (VM) so that it can be allocated to other VMs when the host is running low on memory. A ballon driver inside the VM “inflates” by allocating memory inside the guest OS, forcing the guest OS to release unused pages, which the hypervisor can then reclaim for redistribution.

→ This allows efficient use of physical memory by balancing memory among multiple VMs running on the same host.

→ Overcommitment refers to allocating more resources (memory, CPU) to virtual machines than the actual physical resources available on the host. It allows hosts to run more VMs use their full allocation simultaneously. While increasing utilization and density, overcommitment risks performance degradation if multiple VMs demand peak resources simultaneously.

→ Transparent Page Sharing (TPS) is a hypervisor memory optimization technique that reduces memory usage by sharing identical memory pages among multiple VMS. The hyperviosr scans memory pages of all VMs to detect identical content pages and maps them to a single physical page, saving overall memory. When a VM writes to a shared page, the hypervisor automatically makes a private copy for that VM (copy-on-write). TPS helps improves memory efficiency and allows better resource utilization and overcommitement.

Term	Purpose	Context
Ballooning	Reclaim unused memory from VMs dynamically	Memory management within VMs via balloon driver
Overcommitment	Allocating more resources than physically available	Host-level resource planning for virtualization
Transparent Page Sharing	Share identical memory pages between VMs	Hypervisor-level memory optimization and deduplication